Lab 7: Customer Segmentation and Predictive Analytics Using Machine Learning

Name: Sheshank Singh
Roll Number: 23102A0017
Course: R Programming

Problem Statement

An e-commerce company wants to better understand its customers and improve its marketing decisions. Using historical online retail transaction data, a machine-learning-based analytics solution is developed to identify meaningful customer segments and predict whether a customer belongs to a high-value customer category.

The analysis includes:

- Data preprocessing and cleaning
- Customer-level feature engineering using RFM analysis
- K-Means clustering
- Elbow Method for selecting the number of clusters
- Hierarchical Clustering and dendrogram
- Silhouette Score evaluation
- Principal Component Analysis (PCA)
- 2D and 3D visualization of customer segments
- Customer profiling
- High-value customer classification
- Random Forest classification
- Support Vector Machine (SVM) classification
- Model evaluation using Accuracy, Precision, Recall, F1-Score and ROC-AUC
- Confusion Matrix
- Feature importance analysis
- Segment-wise marketing recommendations

Dataset Description

The dataset used in this experiment is the **Online Retail Dataset** from the UCI Machine Learning Repository.

The dataset contains **541,909 transaction records** from an online retail company. Important attributes include:

Attribute	Description
InvoiceNo	Unique invoice number
StockCode	Product/item code
Description	Product description
Quantity	Number of products purchased
InvoiceDate	Date and time of transaction
UnitPrice	Price per unit
CustomerID	Unique customer identifier
Country	Customer's country

```
# Install required packages
packages <- c(
  "tidyverse",
  "readxl",
  "lubridate",
  "cluster",
  "factoextra",
  "randomForest",
  "e1071",
  "caret",
  "pROC",
  "plotly",
  "scales"
)

installed <- rownames(installed.packages())

for (pkg in packages) {
  if (!(pkg %in% installed)) {
```

```
    install.packages(pkg, repos = "https://cloud.r-project.org")
  }
}

cat("All required packages are ready.\n")
```

```
All required packages are ready.
```

1. Loading the Dataset

The Online Retail dataset is downloaded from the UCI Machine Learning Repository and loaded into R.

The original dataset is provided as an Excel file.

```
# Directly download the Kaggle/UCI-compatible Online Retail dataset
url <- "https://archive.ics.uci.edu/static/public/352/online+retail.zip"

download.file(
  url,
  destfile = "online_retail.zip",
  mode = "wb"
)

# Extract dataset
unzip("online_retail.zip")

# Find Excel file
file_path <- list.files(
  pattern = "\\.xlsx$",
  full.names = TRUE
)[1]

# Read dataset
retail <- readxl::read_excel(file_path)

# Display dataset information
cat("Dataset loaded successfully!\n")
cat("Rows:", nrow(retail), "\n")
cat("Columns:", ncol(retail), "\n")

head(retail)
```

```
Dataset loaded successfully!
Rows: 541909
Columns: 8
```

A tibble: 6 × 8

InvoiceNo <chr>	StockCode <chr>	Description <chr>	Quantity <dbl>	InvoiceDate <dttm>	UnitPrice <dbl>	CustomerID <dbl>	Country <chr>
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850	United Kingdom
536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850	United Kingdom
536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850	United Kingdom
536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850	United Kingdom

2. Initial Data Exploration

Before preprocessing, the structure, data types, missing values and summary statistics are examined.

```
# Dataset structure

str(retail)
```

```
tibble [541,909 × 8] (S3: tbl_df/tbl/data.frame)
 $ InvoiceNo  : chr [1:541909] "536365" "536365" "536365" "536365" ...
 $ StockCode  : chr [1:541909] "85123A" "71053" "84406B" "84029G" ...
 $ Description: chr [1:541909] "WHITE HANGING HEART T-LIGHT HOLDER" "WHITE METAL LANTERN" "CREAM CUPID HEARTS COAT HANGER" "
 $ Quantity   : num [1:541909] 6 6 8 6 6 2 6 6 6 32 ...
 $ InvoiceDate: POSIXct[1:541909], format: "2010-12-01 08:26:00" "2010-12-01 08:26:00" ...
 $ UnitPrice  : num [1:541909] 2.55 3.39 2.75 3.39 3.39 7.65 4.25 1.85 1.85 1.69 ...
 $ CustomerID : num [1:541909] 17850 17850 17850 17850 17850 ...
 $ Country    : chr [1:541909] "United Kingdom" "United Kingdom" "United Kingdom" "United Kingdom" ...
```

```
# Basic summary

summary(retail)
```

```
  InvoiceNo          StockCode         Description          Quantity
 Length:541909      Length:541909      Length:541909      Min.   :-80995.00
 Class :character   Class :character   Class :character   1st Qu.:     1.00
 Mode  :character   Mode  :character   Mode  :character   Median :     3.00
                                                          Mean   :     9.55
                                                          3rd Qu.:    10.00
                                                          Max.   : 80995.00

  InvoiceDate                     UnitPrice          CustomerID
 Min.   :2010-12-01 08:26:00.00   Min.   :-11062.06   Min.   :12346
 1st Qu.:2011-03-28 11:34:00.00   1st Qu.:     1.25   1st Qu.:13953
 Median :2011-07-19 17:17:00.00   Median :     2.08   Median :15152
 Mean   :2011-07-04 13:34:57.16   Mean   :     4.61   Mean   :15288
 3rd Qu.:2011-10-19 11:27:00.00   3rd Qu.:     4.13   3rd Qu.:16791
 Max.   :2011-12-09 12:50:00.00   Max.   : 38970.00   Max.   :18287
                                                      NA's   :135080
   Country
 Length:541909
 Class :character
 Mode  :character
```

```
# Missing values in each column

missing_values <- data.frame(
  Column = names(retail),
  Missing_Count = sapply(retail, function(x) sum(is.na(x))),
  Missing_Percentage = round(
    sapply(retail, function(x) mean(is.na(x)) * 100), 2
  )
)

missing_values
```

A data.frame: 8 × 3

	Column	Missing_Count	Missing_Percentage
	<chr>	<int>	<dbl>
InvoiceNo	InvoiceNo	0	0.00
StockCode	StockCode	0	0.00
Description	Description	1454	0.27
Quantity	Quantity	0	0.00
InvoiceDate	InvoiceDate	0	0.00
UnitPrice	UnitPrice	0	0.00
CustomerID	CustomerID	135080	24.93
Country	Country	0	0.00

3. Data Preprocessing

The following preprocessing steps are performed:

1. Remove transactions with missing Customer ID.
2. Remove cancelled transactions.
3. Remove records with non-positive quantity.
4. Remove records with non-positive unit price.
5. Convert InvoiceDate into the appropriate date-time format.
6. Remove duplicate records.
7. Create a transaction-level monetary value.

Cancelled invoices are identified using invoice numbers beginning with the letter `C`.

```
# Load required libraries
library(dplyr)
library(stringr)

# Convert relevant columns
retail <- retail %>%
```

```
  mutate(
    InvoiceNo = as.character(InvoiceNo),
    CustomerID = as.character(CustomerID),
    InvoiceDate = as.POSIXct(InvoiceDate),
    Quantity = as.numeric(Quantity),
    UnitPrice = as.numeric(UnitPrice)
  )

# Dataset size before cleaning
before_rows <- nrow(retail)

# Remove duplicate records
retail <- retail %>%
  distinct()

# Remove missing Customer IDs
retail_clean <- retail %>%
  filter(!is.na(CustomerID))

# Remove cancelled invoices
retail_clean <- retail_clean %>%
  filter(!str_detect(InvoiceNo, "^C"))

# Remove invalid quantities and prices
retail_clean <- retail_clean %>%
  filter(
    Quantity > 0,
    UnitPrice > 0
  )

# Create transaction-level monetary value
retail_clean <- retail_clean %>%
  mutate(
    TotalPrice = Quantity * UnitPrice
  )

# Dataset size after cleaning
after_rows <- nrow(retail_clean)

cat("Rows before preprocessing:", before_rows, "\n")
cat("Rows after preprocessing:", after_rows, "\n")
cat("Rows removed:", before_rows - after_rows, "\n")
```

```
Rows before preprocessing: 536641
Rows after preprocessing: 392692
Rows removed: 143949
```

```
# Check cleaned dataset

summary(retail_clean)

cat("\nMissing Customer IDs:",
    sum(is.na(retail_clean$CustomerID)), "\n")

cat("Cancelled transactions:",
    sum(str_detect(retail_clean$InvoiceNo, "^C")), "\n")
```

```
  InvoiceNo          StockCode         Description           Quantity
 Length:392692      Length:392692      Length:392692      Min.   :    1.00
 Class :character   Class :character   Class :character   1st Qu.:    2.00
 Mode  :character   Mode  :character   Mode  :character   Median :    6.00
                                                          Mean   :   13.12
                                                          3rd Qu.:   12.00
                                                          Max.   :80995.00
  InvoiceDate                     UnitPrice         CustomerID
 Min.   :2010-12-01 08:26:00.00   Min.   :   0.001   Length:392692
 1st Qu.:2011-04-07 11:12:00.00   1st Qu.:   1.250   Class :character
 Median :2011-07-31 12:02:00.00   Median :   1.950   Mode  :character
 Mean   :2011-07-10 19:13:07.76   Mean   :   3.126
 3rd Qu.:2011-10-20 12:53:00.00   3rd Qu.:   3.750
 Max.   :2011-12-09 12:50:00.00   Max.   :8142.750
   Country            TotalPrice
 Length:392692      Min.   :     0.00
 Class :character   1st Qu.:     4.95
 Mode  :character   Median :    12.45
                    Mean   :    22.63
                    3rd Qu.:    19.80
                    Max.   :168469.60

Missing Customer IDs: 0
Cancelled transactions: 0
```

4. Customer-Level Feature Engineering

Customer-level features are generated from the cleaned transaction data.

The main features are:

- **Recency:** Number of days since the customer's most recent purchase.
- **Frequency:** Number of unique invoices generated by the customer.
- **Monetary Value:** Total amount spent by the customer.
- **Average Transaction Value:** Average spending per invoice.
- **Quantity Purchased:** Total number of products purchased.
- **Purchase Frequency:** Number of transactions relative to the customer's active purchasing period.

These features provide a compact representation of customer purchasing behaviour.

```
# Customer-Level Feature Engineering

# Reference date for calculating Recency
reference_date <- max(retail_clean$InvoiceDate) + 86400

customer_data <- retail_clean %>%
  group_by(CustomerID) %>%
  summarise(
    Recency = as.numeric(
      difftime(
        reference_date,
        max(InvoiceDate),
        units = "days"
      )
    ),

    Frequency = n_distinct(InvoiceNo),

    Monetary = sum(TotalPrice),

    Quantity_Purchased = sum(Quantity),

    Average_Transaction_Value = mean(TotalPrice),

    First_Purchase = min(InvoiceDate),

    Last_Purchase = max(InvoiceDate),

    .groups = "drop"
  ) %>%
  mutate(
    Active_Days = as.numeric(
      difftime(
        Last_Purchase,
        First_Purchase,
        units = "days"
      )
    ),

    Active_Months = pmax(Active_Days / 30, 1),

    Purchase_Frequency = Frequency / Active_Months
  )

# Display customer-level features
head(customer_data)

cat("\nNumber of customers:", nrow(customer_data), "\n")
```

A tibble: 6 × 11

CustomerID	Recency	Frequency	Monetary	Quantity_Purchased	Average_Transaction_Value	First_Purchase	Last_Purchase	A
<chr>	<dbl>	<int>	<dbl>	<dbl>	<dbl>	<dttm>	<dttm>	
12346	326.117361	1	77183.60	74215	77183.60000	2011-01-18 10:01:00	2011-01-18 10:01:00	
12347	2.873611	7	4310.00	2458	23.68132	2010-12-07 14:57:00	2011-12-07 15:52:00	
12348	75.984028	4	1797.24	2341	57.97548	2010-12-16 19:09:00	2011-09-25 13:13:00	
12349	19.124306	1	1757.55	631	24.07603	2011-11-21 09:51:00	2011-11-21 09:51:00	
12350	310.867361	1	334.40	197	19.67059	2011-02-02	2011-02-02	

```
# Remove customers with invalid values

customer_data <- customer_data %>%
  filter(
    is.finite(Recency),
    is.finite(Frequency),
    is.finite(Monetary),
    is.finite(Quantity_Purchased),
    is.finite(Average_Transaction_Value),
    is.finite(Purchase_Frequency)
  )

cat("Number of unique customers:", nrow(customer_data), "\n")

summary(
  customer_data %>%
    select(
      Recency,
      Frequency,
      Monetary,
      Quantity_Purchased,
      Average_Transaction_Value,
      Purchase_Frequency
    )
)
```

```
Number of unique customers: 4338
    Recency         Frequency          Monetary         Quantity_Purchased
 Min.   :  1.00   Min.   :  1.000   Min.   :     3.75   Min.   :     1.0
 1st Qu.: 18.07   1st Qu.:  1.000   1st Qu.:   306.48   1st Qu.:   159.0
 Median : 51.09   Median :  2.000   Median :   668.57   Median :   378.0
 Mean   : 93.05   Mean   :  4.272   Mean   :  2048.69   Mean   :  1187.6
 3rd Qu.:142.73   3rd Qu.:  5.000   3rd Qu.:  1660.60   3rd Qu.:   989.8
 Max.   :374.12   Max.   :209.000   Max.   :280206.02   Max.   :196915.0
 Average_Transaction_Value Purchase_Frequency
 Min.   :    2.14          Min.   : 0.1639
 1st Qu.:   12.39          1st Qu.: 0.6084
 Median :   17.75          Median : 1.0000
 Mean   :   68.38          Mean   : 1.0366
 3rd Qu.:   24.88          3rd Qu.: 1.0000
 Max.   :77183.60          Max.   :34.0000
```

5. Outlier Treatment

Retail transaction data can contain customers with exceptionally large purchase values. Such observations can strongly influence distance-based clustering algorithms.

Therefore, extreme values are capped using the 1st and 99th percentiles.

The original customer-level data is retained separately so that the resulting clusters can later be interpreted using the original values.

```
# Features used for clustering

cluster_features <- c(
  "Recency",
  "Frequency",
  "Monetary",
  "Quantity_Purchased",
  "Average_Transaction_Value",
  "Purchase_Frequency"
)

# Function for percentile-based capping
```

```
cap_outliers <- function(x, lower = 0.01, upper = 0.99) {
  q <- quantile(x, probs = c(lower, upper), na.rm = TRUE)
  x[x < q[1]] <- q[1]
  x[x > q[2]] <- q[2]
  return(x)
}

# Create modelling dataset
customer_model <- customer_data

for (feature in cluster_features) {
  customer_model[[feature]] <- cap_outliers(
    customer_model[[feature]]
  )
}

summary(customer_model[cluster_features])
```

```
    Recency          Frequency         Monetary       Quantity_Purchased
 Min.   :  1.777   Min.   : 1.000   Min.   :   52.2   Min.   :   12.0
 1st Qu.: 18.072   1st Qu.: 1.000   1st Qu.:  306.5   1st Qu.:  159.0
 Median : 51.090   Median : 2.000   Median :  668.6   Median :  378.0
 Mean   : 93.021   Mean   : 4.009   Mean   : 1592.6   Mean   :  922.3
 3rd Qu.:142.731   3rd Qu.: 5.000   3rd Qu.: 1660.6   3rd Qu.:  989.8
 Max.   :369.670   Max.   :30.000   Max.   :19780.5   Max.   :11016.2
 Average_Transaction_Value Purchase_Frequency
 Min.   :  3.173           Min.   :0.1961
 1st Qu.: 12.393           1st Qu.:0.6084
 Median : 17.755           Median :1.0000
 Mean   : 29.596           Mean   :1.0129
 3rd Qu.: 24.879           3rd Qu.:1.0000
 Max.   :366.753           Max.   :3.8338
```

6. Feature Transformation and Standardization

The customer features have different scales. For example, Monetary Value can be much larger than Recency.

Log transformation is applied to reduce skewness, followed by standardization.

Standardization transforms each variable approximately to:

$$Z = \frac{X - \mu}{\sigma}$$

This ensures that variables with larger numerical values do not dominate the distance calculations used by clustering algorithms.

```
# Log transformation
customer_scaled <- customer_model %>%
  select(all_of(cluster_features)) %>%
  mutate(
    across(
      everything(),
      ~ log1p(.)
    )
  )

# Standardization
customer_scaled <- scale(customer_scaled)

# Convert to matrix
customer_scaled <- as.matrix(customer_scaled)

# Check result
head(customer_scaled)
```

A matrix: 6 × 6 of type dbl

Recency	Frequency	Monetary	Quantity_Purchased	Average_Transaction_Value	Purchase_Frequency
1.4834992	-0.9739874	2.7239896	2.5275130	3.63526814	0.11908129
-1.9241160	1.1108240	1.4692242	1.3883755	0.28428522	-0.77985710
0.3722003	0.4039980	0.7491301	1.3513467	1.36483410	-1.15901226
-0.6584035	-0.9739874	0.7307491	0.3564031	0.30396623	0.11908129
1.4468269	-0.9739874	-0.6338342	-0.5251749	0.06430545	0.11908129
-0.1716263	1.2879548	1.0227964	0.2326752	0.54617070	-0.02921361

7. Determining the Optimal Number of Clusters

The Elbow Method is used to identify a suitable number of clusters.

The Within-Cluster Sum of Squares (WCSS) is calculated for different values of K. The point where the reduction in WCSS starts becoming smaller is considered the elbow point.

```
# Elbow Method

k_values <- 2:10

wss <- sapply(k_values, function(k) {
  kmeans(
    customer_scaled,
    centers = k,
    nstart = 20
  )$tot.withinss
})

elbow_data <- data.frame(
  K = k_values,
  WCSS = wss
)

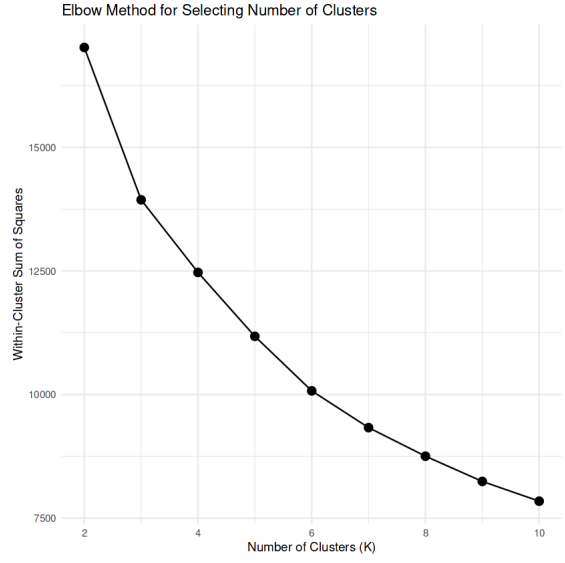
elbow_data
```

A data.frame: 9 × 2

K <int>	WCSS <dbl>
2	17019.955
3	13938.958
4	12470.393
5	11178.235
6	10075.593
7	9331.847
8	8754.122
9	8243.977
10	7844.938

```
# Load ggplot2
library(ggplot2)

# Plot Elbow Method
ggplot(elbow_data, aes(x = K, y = WCSS)) +
  geom_line() +
  geom_point(size = 3) +
  labs(
    title = "Elbow Method for Selecting Number of Clusters",
    x = "Number of Clusters (K)",
    y = "Within-Cluster Sum of Squares"
  ) +
  theme_minimal()
```



Selection of K

Based on the Elbow Method, a suitable value of K is selected. The silhouette analysis below is also used to validate the clustering quality.

For this experiment, the final K is selected using the combined evidence from the Elbow Method and Silhouette Score rather than relying only on the visual elbow.

```
# Calculate average silhouette score for each K
library(cluster)

silhouette_scores <- sapply(k_values, function(k) {

  km <- kmeans(
    customer_scaled,
    centers = k,
    nstart = 20
  )

  sil <- silhouette(
    km$cluster,
    dist(customer_scaled)
  )

  mean(sil[, 3])
})

silhouette_data <- data.frame(
  K = k_values,
  Silhouette_Score = silhouette_scores
)

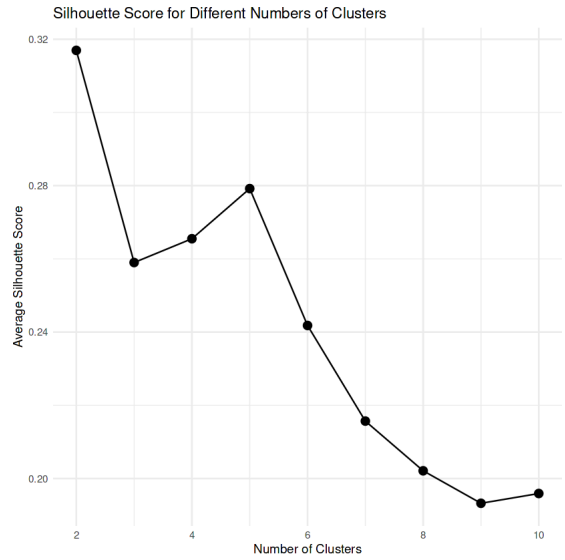
silhouette_data
```

A data.frame: 9 × 2

K <int>	Silhouette_Score <dbl>
2	0.3169523
3	0.2590055
4	0.2655071
5	0.2791970
6	0.2418126
7	0.2156954
8	0.2021211
9	0.1932489
10	0.1959188

```
# Plot silhouette scores

ggplot(
  silhouette_data,
  aes(x = K, y = Silhouette_Score)
) +
  geom_line() +
  geom_point(size = 3) +
  labs(
    title = "Silhouette Score for Different Numbers of Clusters",
    x = "Number of Clusters",
    y = "Average Silhouette Score"
  ) +
  theme_minimal()
```




```
# Select K with the highest silhouette score

optimal_k <- silhouette_data$K[
  which.max(silhouette_data$Silhouette_Score)
]

cat("Selected number of clusters:", optimal_k, "\n")
cat(
  "Maximum Silhouette Score:",
  round(max(silhouette_data$Silhouette_Score), 4),
  "\n"
)
```

```
Selected number of clusters: 2
Maximum Silhouette Score: 0.317
```

8. K-Means Customer Segmentation

K-Means clustering is applied using the selected number of clusters.

The algorithm assigns each customer to the nearest cluster centroid and iteratively updates the centroids until the assignments stabilize.

```
# Final K-Means model

kmeans_model <- kmeans(
  customer_scaled,
  centers = optimal_k,
  nstart = 50,
  iter.max = 100
)

# Add cluster labels
customer_data$KMeans_Cluster <- factor(
  kmeans_model$cluster
)

# Cluster sizes
cluster_sizes <- customer_data %>%
  count(KMeans_Cluster) %>%
  arrange(KMeans_Cluster)

cluster_sizes
```

A tibble: 2 × 2

KMeans_Cluster	n
<fct>	<int>
1	2649
2	1689

```
# Silhouette analysis for final K-Means solution
```

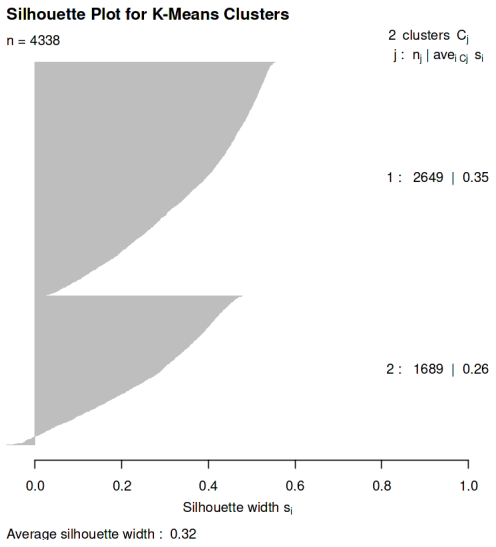
```
final_silhouette <- silhouette(
  kmeans_model$cluster,
  dist(customer_scaled)
)

average_silhouette <- mean(final_silhouette[, 3])

cat(
  "Final Average Silhouette Score:",
  round(average_silhouette, 4),
  "\n"
)

plot(
  final_silhouette,
  main = "Silhouette Plot for K-Means Clusters"
)
```

```
Final Average Silhouette Score: 0.317
```



9. Hierarchical Clustering

Hierarchical clustering is performed to compare the customer segmentation obtained using K-Means.

Ward's method is used because it attempts to minimize the increase in within-cluster variance when clusters are merged.

A sample of customers is used for the dendrogram so that the visualization remains computationally manageable.

```
# Sample customers for hierarchical clustering

set.seed(123)

hier_sample_size <- min(1500, nrow(customer_scaled))

hier_indices <- sample(
  seq_len(nrow(customer_scaled)),
  hier_sample_size
)

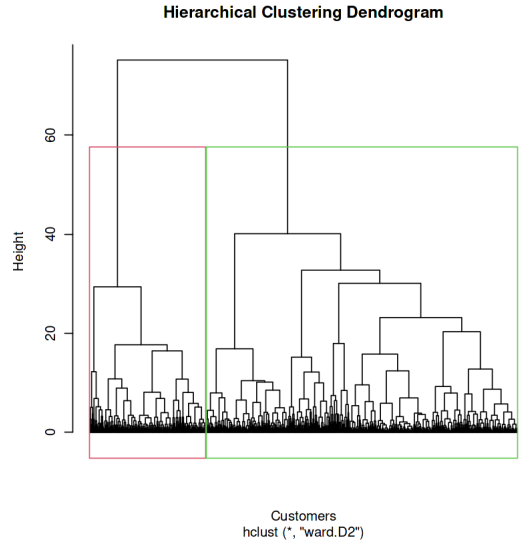
hier_data <- customer_scaled[hier_indices, ]

# Hierarchical clustering
hierarchical_model <- hclust(
  dist(hier_data),
  method = "ward.D2"
)

# Dendrogram
plot(
  hierarchical_model,
  labels = FALSE,
  hang = -1,
  main = "Hierarchical Clustering Dendrogram",
  xlab = "Customers",
  ylab = "Height"
)

# Add cluster boundaries
```

```
rect.hclust(
  hierarchical_model,
  k = optimal_k,
  border = 2:6
)
```



10. Comparison of K-Means and Hierarchical Clustering

K-Means and Hierarchical Clustering use different approaches:

Method	Approach
K-Means	Iteratively assigns observations to cluster centroids
Hierarchical	Builds a hierarchy of nested clusters
K-Means	Requires the number of clusters in advance
Hierarchical	Dendrogram can be used to inspect possible cluster counts

The K-Means solution is used as the primary segmentation because it provides direct cluster assignments for the complete customer population and can be used conveniently for subsequent customer profiling.

11. Principal Component Analysis (PCA)

PCA is applied to reduce the six customer-level features into a smaller number of dimensions.

The first two principal components are used to visualize the customer segments in two dimensions.

PCA is performed only for visualization and dimensionality analysis; the original customer features remain available for interpretation and prediction.

```
# PCA

pca_model <- prcomp(
  customer_scaled,
  center = FALSE,
  scale. = FALSE
)

# Variance explained

pca_variance <- summary(pca_model)$importance

pca_variance
```

A matrix: 3 × 6 of type dbl

	PC1	PC2	PC3	PC4	PC5	PC6
Standard deviation	1.763669	1.036396	0.9818961	0.7469337	0.4821959	0.2466018
Proportion of Variance	0.518420	0.179020	0.1606900	0.0929800	0.0387500	0.0101400
Cumulative Proportion	0.518420	0.697440	0.8581300	0.9511100	0.9898600	1.0000000

```
# Prepare PCA dataframe

pca_data <- data.frame(
```

```
  PC1 = pca_model$x[, 1],
  PC2 = pca_model$x[, 2],
  Cluster = customer_data$KMeans_Cluster
)

# 2D PCA visualization

ggplot(
  pca_data,
  aes(
    x = PC1,
    y = PC2,
    color = Cluster
  )
) +
  geom_point(alpha = 0.6, size = 1.5) +
  labs(
    title = "Customer Segments using PCA",
    x = "Principal Component 1",
    y = "Principal Component 2",
    color = "Cluster"
  ) +
  theme_minimal()
```



12. Interactive 3D Visualization using Plotly

An interactive 3D representation is created using the first three principal components.

The visualization allows customer groups to be explored from different viewing angles.

```
# Suppress non-critical warnings for cleaner notebook output
options(warn = -1)
```

```
# Create 3D PCA dataset
library(plotly)

pca_3d <- data.frame(
  PC1 = pca_model$x[, 1],
  PC2 = pca_model$x[, 2],
  PC3 = pca_model$x[, 3],
  Cluster = customer_data$KMeans_Cluster
)

# Plotly 3D visualization

fig <- plot_ly(
  pca_3d,
  x = ~PC1,
  y = ~PC2,
  z = ~PC3,
  color = ~Cluster,
  type = "scatter3d",
  mode = "markers",
  marker = list(size = 3)
)
```

```
fig <- fig %>%
  layout(
    title = "Interactive 3D PCA Visualization of Customer Segments",
    scene = list(
      xaxis = list(title = "PC1"),
      yaxis = list(title = "PC2"),
      zaxis = list(title = "PC3")
    )
  )

fig
```

13. Customer Profiling

After clustering, each cluster is profiled using the original customer-level features.

The mean and median values are calculated for:

- Recency
- Frequency
- Monetary Value
- Quantity Purchased
- Average Transaction Value
- Purchase Frequency

This helps interpret the purchasing behaviour represented by each cluster.

```
# Cluster profiling

cluster_profile <- customer_data %>%
  group_by(KMeans_Cluster) %>%
  summarise(
    Customers = n(),
    Avg_Recency = mean(Recency),
    Avg_Frequency = mean(Frequency),
    Avg_Monetary = mean(Monetary),
    Avg_Quantity = mean(Quantity_Purchased),
    Avg_Transaction_Value = mean(
      Average_Transaction_Value
    ),
    Avg_Purchase_Frequency = mean(
      Purchase_Frequency
    ),
    .groups = "drop"
  )

cluster_profile
```

A tibble: 2 × 8

KMeans_Cluster	Customers	Avg_Recency	Avg_Frequency	Avg_Monetary	Avg_Quantity	Avg_Transaction_Value	Avg_Purchase_Frequ
<fct>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<
1	2649	131.52727	1.687052	439.7176	255.4258	27.4735	0.967
2	1689	32.70113	8.326229	4572.1711	2649.7211	132.5412	1.145

```
# Median-based cluster profile

cluster_median_profile <- customer_data %>%
  group_by(KMeans_Cluster) %>%
  summarise(
    Customers = n(),
    Median_Recency = median(Recency),
    Median_Frequency = median(Frequency),
    Median_Monetary = median(Monetary),
    Median_Quantity = median(Quantity_Purchased),
    Median_Transaction_Value =
      median(Average_Transaction_Value),
    Median_Purchase_Frequency =
      median(Purchase_Frequency),
    .groups = "drop"
  )

cluster_median_profile
```

A tibble: 2 × 8

KMeans_Cluster	Customers	Median_Recency	Median_Frequency	Median_Monetary	Median_Quantity	Median_Transaction_Value	Med
<fct>	<int>	<dbl>	<int>	<dbl>	<dbl>	<dbl>	
1	2649	90.07083	1	355.56	197	16.93692	
2	1689	18.83056	6	2092.66	1242	19.18195	

14. Identifying the High-Value Customer Segment

A high-value customer is identified using purchasing behaviour.

For this experiment, the cluster with the highest average **Monetary Value** is selected as the high-value segment.

This creates a binary target variable:

- `1` → Customer belongs to the high-value segment
- `0` → Customer belongs to another segment

The target is generated from the clustering stage and is subsequently used for supervised classification.

```
# Identify cluster with highest average Monetary Value

high_value_cluster <- cluster_profile$KMeans_Cluster[
  which.max(cluster_profile$Avg_Monetary)
]

cat(
  "High-value cluster:",
  as.character(high_value_cluster),
  "\n"
)

# Create target variable

customer_data <- customer_data %>%
  mutate(
    High_Value = ifelse(
      KMeans_Cluster == high_value_cluster,
      1,
      0
    ),
    High_Value = factor(
      High_Value,
      levels = c(0, 1),
      labels = c("Non_High_Value", "High_Value")
    )
  )

table(customer_data$High_Value)
```

```
High-value cluster: 2

Non_High_Value     High_Value
          2649           1689
```

15. Preparing Data for Predictive Modelling

Two supervised machine learning models are developed:

1. Random Forest
2. Support Vector Machine (SVM)

The original customer-level behavioural features are used as predictors.

The dataset is divided into training and testing sets using stratified sampling so that both classes are represented in the two subsets.

```
# Predictive Modelling Dataset

model_data <- customer_data %>%
  select(
    Recency,
    Frequency,
    Monetary,
    Quantity_Purchased,
    Average_Transaction_Value,
    Purchase_Frequency,
    High_Value
  )

# Train-test split (80:20) with stratification
set.seed(123)

high_idx <- which(model_data$High_Value == "High_Value")
non_high_idx <- which(model_data$High_Value == "Non_High_Value")

train_high <- sample(
  high_idx,
  size = floor(0.80 * length(high_idx))
)

train_non_high <- sample(
  non_high_idx,
  size = floor(0.80 * length(non_high_idx))
)

train_index <- c(train_high, train_non_high)

train_data <- model_data[train_index, ]
test_data <- model_data[-train_index, ]

cat("Training observations:", nrow(train_data), "\n")
cat("Testing observations:", nrow(test_data), "\n")

cat("\nTraining class distribution:\n")
print(prop.table(table(train_data$High_Value)))

cat("\nTesting class distribution:\n")
print(prop.table(table(test_data$High_Value)))
```

```
Training observations: 3470
Testing observations: 868

Training class distribution:

Non_High_Value     High_Value
     0.6106628      0.3893372

Testing class distribution:

Non_High_Value     High_Value
     0.6105991      0.3894009
```

16. Random Forest Classification

Random Forest is an ensemble learning method that combines multiple decision trees.

It is suitable for this problem because it can model nonlinear relationships between customer purchasing characteristics and the high-value customer category.

The model is evaluated on unseen test data.

```
# Train Random Forest
library(randomForest)
```

```
set.seed(123)

rf_model <- randomForest(
  High_Value ~
```

```
    Recency +
    Frequency +
    Monetary +
    Quantity_Purchased +
    Average_Transaction_Value +
    Purchase_Frequency,
  data = train_data,
  ntree = 300,
  importance = TRUE
)

rf_model
```

```
Call:
 randomForest(formula = High_Value ~ Recency + Frequency + Monetary +      Quantity_Purchased + Average_Transaction_Value +
Purchase_Frequency,      data = train_data, ntree = 300, importance = TRUE)
               Type of random forest: classification
                     Number of trees: 300
No. of variables tried at each split: 2

        OOB estimate of  error rate: 1.93%
Confusion matrix:
               Non_High_Value High_Value class.error
Non_High_Value           2087         32  0.01510146
```

```
# Random Forest Predictions

rf_prob <- predict(
  rf_model,
  newdata = test_data,
  type = "prob"
)[, "High_Value"]

rf_pred <- factor(
  ifelse(
    rf_prob >= 0.5,
    "High_Value",
    "Non_High_Value"
  ),
  levels = levels(test_data$High_Value)
)

# Confusion Matrix

rf_cm <- table(
  Actual = test_data$High_Value,
  Predicted = rf_pred
)

rf_cm
```

```
                Predicted
Actual           Non_High_Value High_Value
  Non_High_Value            516         14
  High_Value                 18        320
```

17. Random Forest Evaluation

The following metrics are extracted:

- Accuracy
- Precision
- Recall
- F1-Score

ROC-AUC is calculated separately using the predicted probabilities.

```
library(pROC)
# Random Forest Metrics

# Extract confusion matrix values
TN <- rf_cm["Non_High_Value", "Non_High_Value"]
FP <- rf_cm["Non_High_Value", "High_Value"]
FN <- rf_cm["High_Value", "Non_High_Value"]
TP <- rf_cm["High_Value", "High_Value"]

# Calculate metrics
rf_accuracy <- (TP + TN) / sum(rf_cm)
```

```
rf_precision <- TP / (TP + FP)

rf_recall <- TP / (TP + FN)

rf_f1 <- 2 * (rf_precision * rf_recall) /
         (rf_precision + rf_recall)

# ROC-AUC
rf_roc <- roc(
  response = test_data$High_Value,
  predictor = rf_prob,
  levels = c("Non_High_Value", "High_Value"),
  direction = "<"
)

rf_auc <- as.numeric(auc(rf_roc))

# Create metrics table
rf_metrics <- data.frame(
  Model = "Random Forest",
  Accuracy = rf_accuracy,
  Precision = rf_precision,
  Recall = rf_recall,
  F1_Score = rf_f1,
  ROC_AUC = rf_auc
)

rf_metrics
```

A data.frame: 1 × 6

Model	Accuracy	Precision	Recall	F1_Score	ROC_AUC
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
Random Forest	0.9631336	0.9580838	0.9467456	0.952381	0.9973931

18. Random Forest Feature Importance

Feature importance is examined to understand which customer characteristics contribute most strongly to the classification model.

```
# Extract feature importance

importance_data <- data.frame(
  Feature = rownames(importance(rf_model)),
  Importance = importance(rf_model)[, "MeanDecreaseGini"]
)

importance_data <- importance_data %>%
  arrange(desc(Importance))

importance_data
```

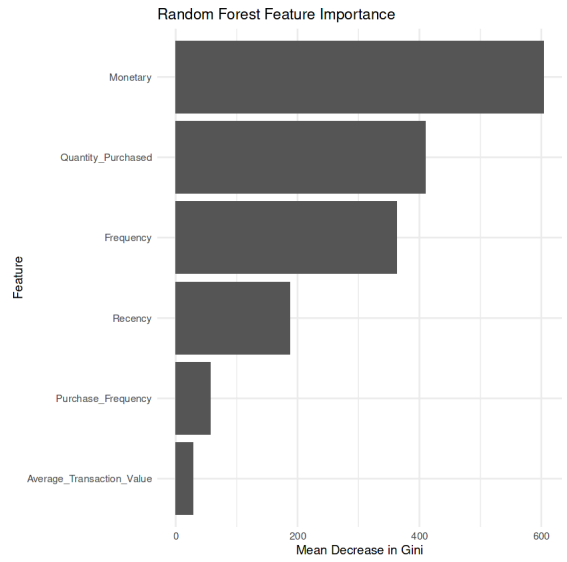
A data.frame: 6 × 2

	Feature	Importance
	<chr>	<dbl>
Monetary	Monetary	603.63571
Quantity_Purchased	Quantity_Purchased	409.80223
Frequency	Frequency	362.90582
Recency	Recency	187.67301
Purchase_Frequency	Purchase_Frequency	56.62790
Average_Transaction_Value	Average_Transaction_Value	28.40859

```
# Plot feature importance

ggplot(
  importance_data,
  aes(
    x = reorder(Feature, Importance),
    y = Importance
  )
) +
  geom_col() +
  coord_flip() +
  labs(
```

```
    title = "Random Forest Feature Importance",
    x = "Feature",
    y = "Mean Decrease in Gini"
  ) +
  theme_minimal()
```



19. Support Vector Machine (SVM)

A Support Vector Machine classifier is developed as the second supervised learning model.

SVM attempts to find a decision boundary that separates the customer classes while maximizing the margin between them.

Feature scaling is particularly important for SVM, therefore the training features are standardized before model fitting.

```
# Prepare feature matrices

predictor_names <- c(
  "Recency",
  "Frequency",
  "Monetary",
  "Quantity_Purchased",
  "Average_Transaction_Value",
  "Purchase_Frequency"
)

# Scale using training statistics

train_x <- train_data[, predictor_names]
test_x <- test_data[, predictor_names]

train_means <- sapply(train_x, mean)
train_sds <- sapply(train_x, sd)

train_x_scaled <- scale(
  train_x,
  center = train_means,
  scale = train_sds
)

test_x_scaled <- scale(
  test_x,
  center = train_means,
  scale = train_sds
)

train_x_scaled <- as.data.frame(train_x_scaled)
test_x_scaled <- as.data.frame(test_x_scaled)

train_x_scaled$High_Value <- train_data$High_Value
test_y <- test_data$High_Value
```

```
# Train SVM
library(e1071)
set.seed(123)
```

```
svm_model <- svm(
  High_Value ~ .,
  data = train_x_scaled,
  kernel = "radial",
  probability = TRUE
)

svm_model
```

```
Call:
svm(formula = High_Value ~ ., data = train_x_scaled, kernel = "radial",
    probability = TRUE)


Parameters:
   SVM-Type:  C-classification
 SVM-Kernel:  radial
       cost:  1

Number of Support Vectors:  735
```

```
# SVM Predictions

svm_pred <- predict(
  svm_model,
  newdata = test_x_scaled,
  probability = TRUE
)

# Extract probabilities

svm_probabilities <- attr(
  svm_pred,
  "probabilities"
)

svm_prob <- svm_probabilities[, "High_Value"]

# Confusion Matrix

svm_cm <- table(
  Actual = test_y,
  Predicted = factor(
    svm_pred,
    levels = levels(test_y)
  )
)

svm_cm
```

```
                Predicted
Actual           Non_High_Value High_Value
  Non_High_Value            512         18
  High_Value                 21        317
```

```
# SVM Metrics

TN <- svm_cm["Non_High_Value", "Non_High_Value"]
FP <- svm_cm["Non_High_Value", "High_Value"]
FN <- svm_cm["High_Value", "Non_High_Value"]
TP <- svm_cm["High_Value", "High_Value"]

svm_accuracy <- (TP + TN) / sum(svm_cm)

svm_precision <- TP / (TP + FP)

svm_recall <- TP / (TP + FN)

svm_f1 <- 2 * (svm_precision * svm_recall) /
          (svm_precision + svm_recall)

# ROC-AUC

svm_roc <- roc(
  response = test_y,
  predictor = svm_prob,
  levels = c("Non_High_Value", "High_Value"),
  direction = "<"
)

svm_auc <- as.numeric(auc(svm_roc))
```

```
# Create metrics table

svm_metrics <- data.frame(
  Model = "SVM",
  Accuracy = svm_accuracy,
  Precision = svm_precision,
  Recall = svm_recall,
  F1_Score = svm_f1,
  ROC_AUC = svm_auc
)

svm_metrics
```

A data.frame: 1 × 6

Model	Accuracy	Precision	Recall	F1_Score	ROC_AUC
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
SVM	0.9550691	0.9462687	0.9378698	0.9420505	0.9948867

20. Model Performance Comparison

The performance of Random Forest and SVM is compared using:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC

Higher values indicate stronger performance for the corresponding metric. The comparison is performed on the same unseen test dataset.

```
# Combine model metrics

model_comparison <- bind_rows(
  rf_metrics,
  svm_metrics
)

# Round values for display

model_comparison_display <- model_comparison %>%
  mutate(
    across(
      where(is.numeric),
      ~ round(.x, 4)
    )
  )

model_comparison_display
```

A data.frame: 2 × 6

Model	Accuracy	Precision	Recall	F1_Score	ROC_AUC
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
Random Forest	0.9631	0.9581	0.9467	0.9524	0.9974
SVM	0.9551	0.9463	0.9379	0.9421	0.9949

21. ROC Curves

ROC curves are plotted for both models to compare their ability to distinguish high-value customers from other customers across different classification thresholds.

The Area Under the Curve (AUC) summarizes the ROC performance.

```
# ROC Curves for Random Forest and SVM

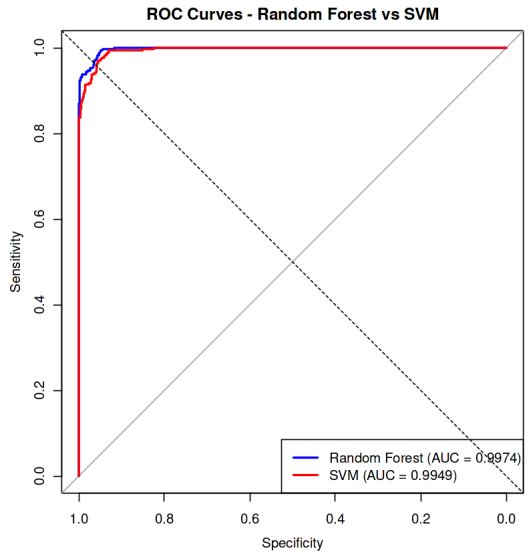
plot(
  rf_roc,
  col = "blue",
  lwd = 2,
  main = "ROC Curves - Random Forest vs SVM"
```

```
)

lines(
  svm_roc,
  col = "red",
  lwd = 2
)

abline(
  a = 0,
  b = 1,
  lty = 2
)

legend(
  "bottomright",
  legend = c(
    paste0("Random Forest (AUC = ", round(rf_auc, 4), ")"),
    paste0("SVM (AUC = ", round(svm_auc, 4), ")")
  ),
  col = c("blue", "red"),
  lwd = 2
)
```




```
# ROC-AUC comparison

auc_comparison <- data.frame(
  Model = c("Random Forest", "SVM"),
  ROC_AUC = c(
    rf_auc,
    svm_auc
  )
)

auc_comparison$ROC_AUC <- round(
  auc_comparison$ROC_AUC,
  4
)

auc_comparison
```

A data.frame: 2 × 2

Model	ROC_AUC
<chr>	<dbl>
Random Forest	0.9974
SVM	0.9949

22. Confusion Matrix Comparison

A confusion matrix provides the number of:

- True Positives
- True Negatives

- False Positives
- False Negatives

It helps understand the types of classification errors made by each model.

```
# Confusion Matrices

cat("Random Forest Confusion Matrix\n")
print(rf_cm)

cat("\nSVM Confusion Matrix\n")
print(svm_cm)
```

```
Random Forest Confusion Matrix
                Predicted
Actual           Non_High_Value High_Value
  Non_High_Value            516         14
  High_Value                 18        320

SVM Confusion Matrix
                Predicted
Actual           Non_High_Value High_Value
  Non_High_Value            512         18
  High_Value                 21        317
```

23. Segment-Wise Customer Characteristics

The final customer segments are examined using their purchasing characteristics.

The profiles are used to understand how customer groups differ in terms of recency, purchase frequency, quantity and monetary value.

```
# Normalized profile for comparison

profile_normalized <- cluster_profile %>%
  mutate(
    Recency_Index = scales::rescale(
      -Avg_Recency,
      to = c(0, 1)
    ),
    Frequency_Index = scales::rescale(
      Avg_Frequency,
      to = c(0, 1)
    ),
    Monetary_Index = scales::rescale(
      Avg_Monetary,
      to = c(0, 1)
    )
  )

profile_normalized
```

A tibble: 2 × 11

KMeans_Cluster	Customers	Avg_Recency	Avg_Frequency	Avg_Monetary	Avg_Quantity	Avg_Transaction_Value	Avg_Purchase_Frequ
<fct>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<
1	2649	131.52727	1.687052	439.7176	255.4258	27.4735	0.967
2	1689	32.70113	8.326229	4572.1711	2649.7211	132.5412	1.145

24. Marketing Recommendations

Based on the purchasing characteristics of each cluster, the following marketing strategies can be considered.

High-Value Customers

Customers in the high-value segment show comparatively high monetary value. Possible strategies include:

- Loyalty rewards
- Premium offers
- Early access to new products
- Personalized recommendations
- Exclusive customer benefits

Frequent Customers

Customers with high purchase frequency can be encouraged through:

- Loyalty points
- Subscription-based offers
- Cross-selling
- Personalized product recommendations

At-Risk Customers

Customers with high recency values have not purchased recently. Suitable strategies include:

- Re-engagement campaigns
- Limited-time discounts
- Reminder emails
- Personalized offers

Low-Value / Occasional Customers

Customers with comparatively low purchase activity can be targeted through:

- Introductory offers
- Product recommendations
- Bundle discounts
- Seasonal campaigns

The actual strategy should be selected according to the numerical profile of each cluster rather than assuming that every cluster has the same behaviour.

25. Conclusion

This experiment demonstrated an end-to-end customer analytics workflow using the Online Retail dataset.